Network Security Definitions (Chapter 8)

8.1 & 8.2 Principles of Cryptography & Attacks

- Only sender and intended receiver can understand the message (encryption).
- Confirming the identity of the communicating party.
- Ensuring message content has not been altered (hashes/MACs).
- Ensuring services are accessible (defending against DoS).
  - **Cipher-text only attack:** Intruder has access only to the intercepted ciphertext. Uses **Brute Force** (try all keys) or **Statistical Analysis** (letter frequency).
  - **Known-plaintext attack:** Intruder knows some plaintext/ciphertext pairings (e.g., knows Alice always starts emails with "Dear Bob").
  - **Chosen-plaintext attack:** Intruder can choose specific plaintext and get the corresponding ciphertext generated by the target.

Shared secret key. Fast.

- **DES (Data Encryption Standard):** Older standard. 56-bit key (too small/insecure). Block cipher.
- **AES (Advanced Encryption Standard):** Modern standard. 128, 192, or 256-bit keys.

Key pairs (Public/Private). Solves key distribution. Slower.

- **RSA:** Security based on difficulty of factoring large numbers ($n = pq$).

Input $m \rightarrow$ fixed size digest $H(m)$.

- **Properties:** Many-to-1 map; fixed size output (fingerprint); computationally infeasible to find m given x (one-way); collision resistant.
- **MD5:** 128-bit digest (older, broken collision resistance).
- **SHA-1:** 160-bit digest (US standard, phasing out).
- **SHA-256:** 256-bit digest (modern standard).

8.3 Authentication Protocol Evolution

The iterative process of designing a secure authentication protocol (ap).
ap1.0 (I am Alice) Vulnerable to impersonation/spoofing.
ap2.0 (Source IP) Vulnerable to IP spoofing.
ap3.0 (Password) Vulnerable to eavesdropping (sniffing) and playback attacks.
ap3.1 (Encrypted Password) Vulnerable to recording the encrypted password and playing it back (Playback Attack).
ap4.0 (Nonces) Uses a "number used once" (R) to prove aliveness. Vulnerable if symmetric keys aren't established securely.
ap5.0 (Public Key Nonces) Uses Public Key crypto to sign nonces. Vulnerable to **Man-in-the-Middle** attacks unless Public Keys are certified.

8.4 Securing Email

Digital Signature Process Provides Integrity, Authentication, and Non-Repudiation.
1. Sender computes Hash $H(m)$ of message.
2. Sender encrypts $H(m)$ with Sender's **Private Key** to create Signature.
3. Receiver decrypts Signature with Sender's **Public Key** and compares to calculated hash.

Confidentiality Only Sender generates random symmetric session key K_S. Encrypts message with K_S. Encrypts K_S with Receiver's **Public Key**.
Full Secure Email (Confidentiality + Integrity + Auth)
1. Alice generates random session key K_S.
2. Alice signs the message (Hash + Private Key).
3. Alice encrypts (Message + Signature) using K_S.
4. Alice encrypts K_S using Bob's **Public Key**.
5. Alice sends the package.

8.5 Securing TCP: TLS (Transport Layer Security)

TLS Services Confidentiality (Symmetric encryption), Integrity (Cryptographic Hashing), Authentication (Public Key certificates). Successor to SSL.
TLS Phases 1. Handshake, 2. Key Derivation, 3. Data Transfer, 4. Connection Closure.
TLS Handshake Phase Sender establishes TCP connection; verifies receiver's identity (Certificates); sends **Master Secret (MS)** to generate all other keys.
Derived Cryptographic Keys It is bad practice to use the same key for multiple functions. TLS derives 4 distinct keys from the MS using a Key Derivation Function (KDF):

- K_c: Encryption key for data sent from Client to Server.
- M_c: MAC key for data sent from Client to Server.
- K_s: Encryption key for data sent from Server to Client.
- M_s: MAC key for data sent from Server to Client.

TLS Data Transfer (Records) TCP is a byte stream; TLS breaks this into "Records".
- **Process:** Calculate MAC using M_c. Append MAC to data. Encrypt (Data+MAC) using K_c. Add header.
- **Result:** K_c(Length | Data | MAC).

Attacks on Records & Solutions
- **Re-ordering Attack:** MITM reorders TCP segments.
- **Solution:** Use TLS Sequence Numbers (incorporated into the MAC calculation, not sent explicitly).
- **Replay Attack:** MITM captures and resends records.
- **Solution:** Use Nonces.
- **Truncation Attack:** Attacker forges a TCP FIN to close connection early.
- **Solution: Record Types**. Type 0 for data, Type 1 for closure. The closure message is authenticated (MAC'd) so it cannot be forged.

TLS 1.3 Updates
- **Cipher Suites:** Limited to 5 choices (vs 37 in TLS 1.2).
- **Diffie-Hellman (DH):** Required for key exchange (RSA removed for key exchange to ensure forward secrecy).
- **Algorithms:** Uses "Authenticated Encryption" (combined encryption+integrity). 4 of 5 suites based on AES. HMAC uses SHA-256 or SHA-384.

TLS 1.3 Handshake: 1-RTT
1. **Client Hello:** Guesses key agreement protocol/params; sends supported cipher suites; sends "key share" (half of DH).
2. **Server Hello:** Chooses key agreement/cipher suite; sends server-signed certificate; sends its "key share".
3. **Client:** Checks cert; generates keys; can now send encrypted application request.

TLS 1.3 Handshake: 0-RTT (Resumption)
- Used when resuming a previous session.
- **Client Hello** includes encrypted application data immediately!
- Encrypted using a **"Resumption Master Secret"** derived from the previous session.
- **Risk:** Vulnerable to **Replay Attacks** because the first message isn't challenged by the server before being processed.

TLS record general format

Byte	+0	+1	+2	+3
0	Content Type	N/A		
1..4	Legacy version		Length	
5..n	*Payload*			
n..m	MAC			
m..p	Padding (block ciphers only)			

